Week 10 – Interview Preparation – Angular Track

Code Analysis and Logging, Debugging, Angular

1. What is code analysis in .NET, and why is it important?
Code analysis examines source code for potential errors, code smells, and rule violations. It helps maintain quality, enforce standards, and catch issues early in development.

2. How do you enable code analysis in a .NET project?
You can enable it via project settings (.csproj) using <EnableNETAnalyzers>true</EnableNETAnalyzers> and <AnalysisLevel>latest</AnalysisLevel>, or through Visual Studio settings.

---

3. What are code metrics?
Code metrics are quantitative measures (like complexity, maintainability, coupling) used to evaluate code quality.

4. Name some common code metrics in .NET.
Cyclomatic complexity, maintainability index, depth of inheritance, and class coupling.

5. Why is cyclomatic complexity important?
It measures the number of independent execution paths in code. Higher values indicate more complex and harder-to-test code.

---

6. What are code analyzers in .NET?
They are tools that inspect code and provide diagnostics (warnings/errors) based on predefined or custom rules.

7. What is the difference between a warning and an error in diagnostics?
Warnings indicate potential issues but don't stop compilation; errors must be fixed before the build succeeds.

---

8. What are some common code analysis rule categories?
Design, performance, security, maintainability, and naming conventions.

9. How can you suppress a code analysis warning?
Using #pragma warning disable, attributes like [SuppressMessage], or configuration in .editorconfig.

---

10. How does code analysis help in code reviews?
It automates detection of common issues, allowing reviewers to focus on logic, design, and business requirements.

11. Should code analysis replace manual code reviews? Why or why not?
No, it complements reviews but cannot assess business logic, architecture, or intent.

---

12. What is logging in .NET?
Logging is the process of recording application events, errors, and diagnostics for monitoring and debugging.

13. What logging framework is built into .NET 8?
The built-in Microsoft.Extensions.Logging framework.

---

14. How do you configure logging in .NET 8?
Logging is configured in Program.cs using builder.Logging or via appsettings.json.

---

15. What are the standard logging levels in .NET?
Trace, Debug, Information, Warning, Error, Critical, and None.

16. What is a logging category?
A category typically represents the source (e.g., class name) of the log message.

---

17. How do you use logging in a controller?
Inject ILogger<T> into the controller and use methods like LogInformation() or LogError().

18. Why is logging important in services?
It helps track business operations, diagnose issues, and monitor system behavior.

---

19. How can you filter logs in .NET?
By configuring log levels per category in appsettings.json or programmatically in Program.cs.

---

20. What is log4net, and how is it used?
log4net is a third-party logging framework. It is configured via XML and used to log messages with different appenders (file, console, etc.).

---

21. What is Web API debugging and why is it important?
Web API debugging is the process of identifying and fixing issues in API endpoints, request handling, and responses. It ensures reliability, correct data flow, and proper error handling.

22. What are common issues encountered while debugging Web APIs?
Incorrect routing, model binding failures, null references, authentication issues, and improper HTTP status codes.

---

23. What are breakpoints and how are they used in debugging?
Breakpoints pause execution at a specific line, allowing developers to inspect variables and execution flow.

24. What is the difference between Step Into, Step Over, and Step Out?

- Step Into: Goes inside method calls
- Step Over: Executes method without entering
- Step Out: Exits the current method

25. What is the Watch window in Visual Studio?
It allows you to monitor variables and expressions during debugging.

---

26. What is exception handling in Web APIs?
It is the process of catching and managing runtime errors to prevent application crashes and return meaningful responses.

27. How can you handle global exceptions in ASP.NET Web API?
Using middleware, exception filters, or built-in exception handling mechanisms like UseExceptionHandler.

---

28. What is Postman and why is it used?
Postman is a popular API testing tool used to send HTTP requests, inspect responses, and automate API testing.

29. What types of HTTP requests can be tested using Postman?
GET, POST, PUT, DELETE, PATCH, and more.

---

30. How can Postman help in debugging API calls?
It allows you to inspect request headers, body, parameters, and response status, helping identify issues quickly.

31. What are common mistakes detected using Postman?
Incorrect endpoints, wrong HTTP methods, invalid headers, authentication failures, and malformed JSON.

---

32. What is logging's role in debugging APIs?
Logging provides runtime insights, helping trace execution flow and identify issues in production environments.

33. What is remote debugging?
Remote debugging allows developers to debug applications running on a different environment (e.g., server or container).

34. What are diagnostic tools in .NET for advanced debugging?
Tools like Performance Profiler, dotMemory, and built-in diagnostics for memory, CPU, and threading analysis.

---

35. What is Angular and why is it used?
Angular is a front-end framework for building dynamic, single-page applications (SPAs) using TypeScript. It provides features like two-way data binding, dependency injection, and modular architecture.

36. What are the prerequisites for setting up an Angular development environment?
You need Node.js, npm (Node Package Manager), and the Angular CLI installed globally.

37. What is Angular CLI and how do you install it?
Angular CLI is a command-line tool used to create, build, and manage Angular applications. It can be installed using npm install -g @angular/cli.

---

38. What is Node.js and where is it used?
Node.js is a JavaScript runtime built on Chrome's V8 engine, used for building scalable server-side and networking applications.

39. What are the key features of Node.js?
Non-blocking I/O, event-driven architecture, single-threaded model, and high scalability.

---

40. What are core modules in Node.js?
Core modules are built-in modules provided by Node.js without requiring installation.

41. Name some commonly used Node.js core modules.
Examples include http, fs, path, os, and events.

42. What is the purpose of the 'fs' module?
The fs (File System) module is used to read, write, update, and delete files.

---

43. What is asynchronous programming in Node.js?
It allows tasks to run without blocking the main thread, improving performance and scalability.

44. What are callbacks in Node.js?
Callbacks are functions passed as arguments to be executed after an asynchronous operation completes.

45. What are Promises and how do they improve over callbacks?
Promises represent a future value of an asynchronous operation and help avoid callback nesting (callback hell).

46. What is async/await?
It is a syntax built on Promises that allows writing asynchronous code in a synchronous-like manner, improving readability.

47. What is TypeScript and why is it used in Angular?
TypeScript is a superset of JavaScript that adds static typing and modern features. Angular uses it for better tooling, type safety, and maintainability.

48. What are basic data types available in TypeScript?
Common types include number, string, boolean, any, unknown, void, null, and undefined.

49. What is type inference in TypeScript?
Type inference allows TypeScript to automatically determine the type of a variable based on its value.

---

50. What is the difference between regular functions and arrow functions in TypeScript?
Arrow functions have a shorter syntax and do not bind their own this, instead inheriting it from the surrounding scope.

51. When should you use arrow functions?
They are useful in callbacks and when you want to preserve the lexical this context.

---

52. What is an interface in TypeScript?
An interface defines the structure of an object, specifying properties and their types.

53. What is a type alias and how is it different from an interface?
A type alias defines a custom type using type. Unlike interfaces, it can represent primitives, unions, and intersections.

54. Can interfaces extend other interfaces?
Yes, interfaces can extend one or multiple interfaces to promote reusability.

---

55. How are classes implemented in TypeScript?
Classes are defined using the class keyword and support constructors, properties, methods, and access modifiers.

56. What are access modifiers in TypeScript?
public, private, and protected control visibility of class members.

57. What is inheritance in TypeScript?
Inheritance allows a class to reuse properties and methods of another class using the extends keyword.

58. What is polymorphism in TypeScript?
Polymorphism allows methods to behave differently based on the object instance, often achieved via method overriding or interfaces.

---

59. What are generics in TypeScript?
Generics allow you to create reusable components that work with different data types while maintaining type safety.

60. Why are generics useful?
They improve code reusability, flexibility, and type safety without sacrificing performance.

61. How do you define a generic function in TypeScript?
By using angle brackets: function identity<T>(arg: T): T { return arg; }

62. What are generic constraints?
They restrict the types that can be used with generics using extends.

63. Can you use multiple generic types in a function or class?
Yes, you can define multiple type parameters like <T, U>.

---

64. What are modules in TypeScript?
Modules help organize code into separate files and enable reuse through exports and imports.

65. What is the difference between named and default exports?
Named exports allow multiple exports per file, while default export allows only one.

66. How do you import all exports from a module?
Using import * as moduleName from 'module';

67. What is the purpose of tsconfig.json in module resolution?
It defines compiler options such as module type, base URL, and path mappings.

---

68. What are enums in TypeScript?
Enums are a way to define a set of named constants.

69. What is the difference between numeric and string enums?
Numeric enums auto-increment values, while string enums have explicit string values.

70. What are utility types in TypeScript?
Predefined generic types like Partial, Required, Readonly, and Pick that help transform types.

71. What does the Partial<T> utility type do?
It makes all properties of a type optional.

72. What is the use of Readonly<T>?
It makes all properties immutable.

---

73. How does Angular leverage TypeScript features?
Angular uses TypeScript for decorators, dependency injection, interfaces, and strong typing.

74. What are decorators in Angular?
Decorators are functions that add metadata to classes, properties, or methods (e.g., @Component).

75. What is the role of interfaces in Angular applications?
Interfaces define data models and ensure type safety across components and services.

76. How does TypeScript improve Angular development?
It provides better tooling, compile-time checks, and easier refactoring.

---

77. What is a component in Angular?
A component is the building block of an Angular application, consisting of a template, styles, and logic.

78. What are the key parts of a component?
Template (HTML), class (logic), and metadata (decorator).

79. What is data binding in Angular?
It is the synchronization between the model and the view (e.g., interpolation, property binding).

80. What is the difference between one-way and two-way data binding?
One-way binding flows data in one direction; two-way binding synchronizes data between view and model.

81. What are lifecycle hooks in Angular?
Methods like ngOnInit, ngOnDestroy that allow you to hook into component lifecycle events.

82. What is the purpose of ngOnInit?
It is used for initialization logic after component creation.

83. What is component communication in Angular?
It refers to how components share data using @Input, @Output, and services.

---

84. What is change detection in Angular?
It is the mechanism that updates the DOM when application state changes.

85. What is the difference between Default and OnPush change detection strategies?
Default checks all components; OnPush checks only when input references change, improving performance.

86. What is a standalone component in Angular?
A component that does not require an NgModule and can be used independently.

87. What is the role of templates in Angular components?
Templates define the UI using HTML with Angular directives and bindings.